AI ACADEMY · LEVEL 5 · GRADE 10 · WEEK 18 · STUDENT LESSON

Debugging shapes and gradients

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

Before you start

Why does running the same code twice and getting the same answer not establish correctness?

Plan time to read, investigate and explain. Keep predictions separate from observations.

STUDENT LESSON · 1 OF 18

Your mission

> Start with the target: Check dimension contracts before computing a layer. Try the recall before opening notes.

Your learning target

I can check dimension **contract**s before computing a layer.

I can locate the first wrong value in a forward or gradient trace.

I can verify a repair with independent checks and limited claims.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Initialisation and reproducibility. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** and validation curves. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** a small network. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Check dimension **contract**s before computing a layer

Locate the first wrong value in a forward or gradient trace

Verify a repair with independent checks and limited claims

Retrieve what you already know

> Why does running the same code twice and getting the same answer not establish correctness?

My starting explanation

This week's end goal

By the end, I can explain debugging **shape**s and gradients using a traced example and a stated limitation.

Notice the evidence

> Investigate the supplied Debugging shapes and gradients case. Separate observations, given facts and assumptions before choosing an explanation.

For this lesson, a dense layer stores one **output** unit per weight-matrix row. If an **input** has two features and the layer has three units, W has three rows and two columns, b has length 3, and one **input** x has length 2.

Compute z for each unit, then apply ReLU to each component.

Use W rows [1, 0], [0, 1], [1, -1] and b = [0, 0, 1]. For x = [1, 2], the scores are [1, 2, 0] and the activations are [1, 2, 0].

A batch stores examples in rows: **input**s [[1, 2], [3, 1]] have **shape** 2 by 2, and **output**s have **shape** 2 by 3.

Object	Declared shape	Meaning
One input x	Length 2	Two features
Weights W	3 by 2	Three units each reading two features
Bias b	Length 3	One bias per unit
Batch output	2 by 3	Two examples each with three activations

Other software may use a transposed weight convention. The key is to record and follow one **contract**. An array with the right number of values can still be wrong if features or axes are swapped.

> **List the given facts and one fact that is not supplied.**

See how it works

> Follow the mechanism for Debugging shapes and gradients. Connect each labelled input, deciding step and result.

Python zip stops when its shortest **input** ends. Without a length check, a missing feature can silently remove a contribution.

Reject malformed dimensions before computing: do not pad, truncate or re**shape** a mismatched **input** just to make the calculation run.

```
def dense_relu(x, weights, bias):
    if not x or not weights:
        raise ValueError("Need input features and units")
    if len(bias) != len(weights):
        raise ValueError("One bias per unit required")
    if any(len(row) != len(x) for row in weights):
        raise ValueError("Each unit needs one weight per feature")
    scores = [sum(w * value for w, value in zip(row, x)) + b
              for row, b in zip(weights, bias)]
    return scores, [max(0.0, z) for z in scores]

weights = [[1, 0], [0, 1], [1, -1]]
bias = [0, 0, 1]
for x in [[1, 2], [3, 1]]:
    print(dense_relu(x, weights, bias))
# ([1, 2, 0], [1, 2, 0])
# ([3, 1, 3], [3, 1, 3])
```

The function assumes finite numeric values; it checks dimensions, not every possible **input** fault. Evaluate the two batch rows independently and stack their **output**s.

Retain tests that reject a missing **input** feature, a short weight row and an incorrect bias count.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

STUDENT LESSON · 4 OF 18

Compare the cases

> Compare the supplied cases for Debugging shapes and gradients. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

Compute scores and activations for x = [3, 1] with the lesson layer, then omit the third bias. Identify the first discrepancy and the resulting activation error.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Change one thing in the pictured case

> **Change one thing**
>
> Change: The program changes its declared order to [c, a] and returns [-12,-8].

> **Predicted result**
>
> **Prediction**: That vector now matches the declared coordinate order.

Why the result follows

Reason: Gradient values are meaningful only when their parameter names and array positions agree.

STUDENT LESSON · 5 OF 18

Words that unlock the idea

> Use the vocabulary of Debugging shapes and gradients to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
Shape	The sizes of an array along its declared axes
Contract	A stated condition that inputs and outputs must satisfy
Partial gradient	The change rate of a scalar loss with respect to one parameter while others are fixed
Regression check	A saved check rerun after a repair to detect the return of a defect

Use two terms to explain a step in this week's example.

STUDENT LESSON · 6 OF 18

Follow a worked example

> Follow the worked trace for Debugging shapes and gradients, predict a step, then compare with its explanation.

Repair from the first disagreement

TRACE STAGE FOR X [1 2]	EXPECTED RESULT	SUBMITTED RESULT
Input and dimensions	x length 2 W 3 by 2 b length 3	Dimensions match
Scores before ReLU	[1, 2, 0]	[1, 2, -1]
Hidden activations	[1, 2, 0]	[1, 2, 0]

The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At x = [3, 1], the same omission changes the third activation from 3 to 2.

Trace x = [0, 2] with the correct layer. Then omit the third bias. Which scores differ, and why do the activation vectors still agree?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Debugging shapes and gradients

Catch a gradient -order bug before the optimiser changes parameters.

The facts we will use

> **Supplied facts**
>
> Toy $y=2a+3c$, target 4, start $[a, c]=[1, 0]$. Loss $(y-4)^2$. Program claims gradient $[-12,-8]$ in order $[a, c]$.

> **Supplied facts**
>
> All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

AI ACADEMY / AI-5/18

Debugging shapes and gradients

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Compute correct gradient

y = 2, residual -2: gradient [-8,-12], not [-12,-8].



2. Check coordinates

Step 0.1: a losses 3.24, 4.84 give -8; c losses 2.89, 5.29 give -12.



3. Apply correct update

At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

CHANGE ONE THING

The program changes its declared order to [c, a] and returns [-12,-8].

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Compute correct gradient	y = 2, residual -2: gradient [-8,-12], not [-12,-8].
2. Check coordinates	Step 0.1: a losses 3.24, 4.84 give -8; c losses 2.89, 5.29 give -12.
3. Apply correct update	At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

Array **shape** and coordinate meaning both matter. A vector with two entries can have the right length but assign each derivative to the wrong parameter.

Label the entries a and c, then perturb one coordinate at a time to check their slopes. Repairing the coordinate mapping is necessary before using the gradient to update the network.

What this example does not establish

Keep the stated limit

A correct gradient at one point does not establish every batch, **shape** or activation path is

implemented correctly.

Keep the stated limit

Read the labelled picture: What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Keep the stated limit

Change one thing: The program changes its declared order to [c, a] and returns [-12,-8]. Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the Debugging shapes and gradients claim and explain a repair using the supplied evidence.

Use a separate scalar diagnostic model y = 2w + b, target 3, loss (y - 3) squared, and parameter order [w, b]. At [1, 0], **prediction** is 2, residual -1 and gradient [-4, -2].

A submitted gradient [-2, -4] has the correct length but swapped meanings.

Check the claim against the facts

With symmetric step 0.1, probe w while b stays 0: losses at w = 0.9 and 1.1 are 1.44 and 0.64. Their difference divided by 0.2 is -4. Probe b while w stays 1: losses at b = -0.1 and 0.1 are 1.21 and 0.81, giving -2.

Use original parameters for both probes and compare by coordinate with a stated tolerance.

Check the claim against the facts

A gradient check must include the actual loss convention and any penalty. For ReLU, avoid probes crossing its corner at zero; a symmetric value there need not be a derivative.

A passing numerical check at selected points is **evidence** about those points, not a proof for every **input**.

Locate the earliest unsupported step

For the diagnostic model at [1, 0], gradient [-4, -2] and rate 0.05, calculate the correct new parameters and loss. Compare with adding the gradient step.

State one repair and the checks you would rerun.

STUDENT LESSON · 8 OF 18

Practise together

Practise Debugging shapes and gradients with support, then explain the deciding step in your own words.

The layer expects W **shape** 3 by 2, b length 3 and x length 2. A submitted x has length 3. Should the last feature be silently ignored? State the failed **contract** and repair **process**.

Trace x = [0, 2] with the correct layer. Then omit the third bias. Which scores differ, and why do the activation vectors still agree?

Explain your prediction to a partner before comparing results.

Practise with less help: Expose a hidden bias error

Use workbook W2 for the connected example “Debugging **shape**s and gradients”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Debugging **shape**s and gradients. Proposed explanation: “Matching **output shape** guarantees the computation”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

> **My explanation using a pictured step and the given facts**

This is supported practice. Keep the separately named independent case for an attempt before feedback.

STUDENT LESSON · 9 OF 18

Run the investigation

> Investigate Debugging shapes and gradients. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

Compute scores and activations for x = [3, 1] with the lesson layer, then omit the third bias. Identify the first discrepancy and the resulting activation error.

```
def dense_relu(x, weights, bias):
    if not x or not weights:
        raise ValueError("Need input features and units")
    if len(bias) != len(weights):
        raise ValueError("One bias per unit required")
    if any(len(row) != len(x) for row in weights):
        raise ValueError("Each unit needs one weight per feature")
    scores = [sum(w * value for w, value in zip(row, x)) + b
              for row, b in zip(weights, bias)]
    return scores, [max(0.0, z) for z in scores]
```

```
weights = [[1, 0], [0, 1], [1, -1]]
bias = [0, 0, 1]
for x in [[1, 2], [3, 1]]:
    print(dense_relu(x, weights, bias))
# ([1, 2, 0], [1, 2, 0])
# ([3, 1, 3], [3, 1, 3])
```

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in Debugging shapes and gradients. Explain why each step is justified.

Explain → apply → challenge

The layer expects W **shape** 3 by 2, b length 3 and x length 2. A submitted x has length 3. Should the last feature be silently ignored? State the failed **contract** and repair **process**.

For the diagnostic model at [1, 0], gradient [-4, -2] and rate 0.05, calculate the correct new parameters and loss. Compare with adding the gradient step.

A program passes only the x = [1, 2] final-activation test. Propose one intermediate-value check, one malformed-**input** check and one additional **input** that exposes the bias omission.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Image-pattern Network Investigation. Keep an annotated decision and the evidence supporting it.

Add a labelled debugging **shape**s and gradients **evidence** sheet to Image-pattern Network Investigation. Show how you can check dimension **contract**s before computing a layer.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	

Project evidence	Record
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

At [w, b] = [1, 0], use the verified gradient [-4, -2] and rate 0.05. Subtracting rate times gradient gives [1.2, 0.1], **prediction** 2.5 and loss 0.25. A sign bug that adds it gives [0.8, -0.1], **prediction** 1.5 and loss 2.25.

Recompute loss after the complete update instead of trusting an intended direction.

> **A repair makes one final output match but leaves the third score wrong. Another repair makes loss decrease but has swapped gradient entries. Are either of these sufficient implementation checks? Describe the evidence needed.**

Lower loss alone cannot prove a correct gradient or update; some wrong steps also improve loss. Retain **shape** rejections, hand-computed intermediate traces, numerical coordinate checks and update checks.

Next week investigates the representations produced by hidden layers.

A program passes only the x = [1, 2] final-activation test. Propose one intermediate-value check, one malformed-**input** check and one additional **input** that exposes the bias omission.

> **My limited claim and a useful next check**

STUDENT LESSON · 13 OF 18

Study a complete case

> Revisit the complete Debugging shapes and gradients case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

Repair from the first disagreement

TRACE STAGE FOR X [1 2]	EXPECTED RESULT	SUBMITTED RESULT
Input and dimensions	x length 2 W 3 by 2 b length 3	Dimensions match
Scores before ReLU	[1, 2, 0]	[1, 2, -1]
Hidden activations	[1, 2, 0]	[1, 2, 0]

The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At x = [3, 1], the same omission changes the third activation from 3 to 2.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

Try a new case independently

> Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Debug a new gradient independently

Use y = 3a + c, target 2, loss (y - 2) squared, at [a, c] = [0, 1]. A program returns gradient [-2, -6] in order [a, c].

Use symmetric step 0.1 to check each coordinate, then update both with the corrected gradient and rate 0.05. Report new parameters and loss.

> **My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Use y = 3a + c, target 2, loss (y - 2) squared, at [a, c] = [0, 1]. A program returns gradient [-2, -6] in order [a, c].

Use symmetric step 0.1 to check each coordinate, then update both with the corrected gradient and rate 0.05. Report new parameters and loss.

> **My independent procedure, result and limitation**

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

STUDENT LESSON · 15 OF 18

Your lesson route

> Use this map to navigate Debugging shapes and gradients, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: Catch a gradient -order bug before the optimiser changes parameters.

By the end, I can	How I will show it
Check dimension contracts before computing a layer	Explain the deciding step, show a trace or test, and state one limit.
Locate the first wrong value in a forward or gradient trace	Explain the deciding step, show a trace or test, and state one limit.
Verify a repair with independent checks and limited claims	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain Debugging shapes and gradients without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Check dimension **contract**s before computing a layer

> **My deciding step, evidence and one remaining question**

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Catch a gradient -order bug before the optimiser changes parameters.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

1. Compute correct gradient

2. Check coordinates

3. Apply correct update

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Apply correct update At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904. Array **shape** and coordinate meaning both matter.

A vector with two entries can have the right length but assign each derivative to the wrong parameter. Label the entries a and c, then perturb one coordinate at a time to check their slopes.

Repairing the coordinate mapping is necessary before using the gradient to update the network.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: The program changes its declared order to [c, a] and returns [-12,-8]. Which conclusion is supported by the supplied facts?

1. This one example guarantees correct results for every future **input**.

2. That vector now matches the declared coordinate order.

3. We can decide the result without examining the changed **input** or the procedure.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: That vector now matches the declared coordinate order. Gradient values are meaningful only when their parameter names and array positions agree.

Repair a missing building block

Review Initialisation and reproducibility (ai-5/17). Goal: Trace how starting values affect a fixed **training** budget. Short practice: Does the matching final loss mean the parameter values are equal?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** and validation curves (ai-5/15). Goal: Compute comparable **training** and validation losses at saved checkpoints. Short practice: Can the original epoch 3 **training** loss 1 justify claiming best generalisation?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** a small network (ai-5/12). Goal: Trace repeated forward loss gradient and update steps. Short practice: Which parameter changed, and which part was not trained?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

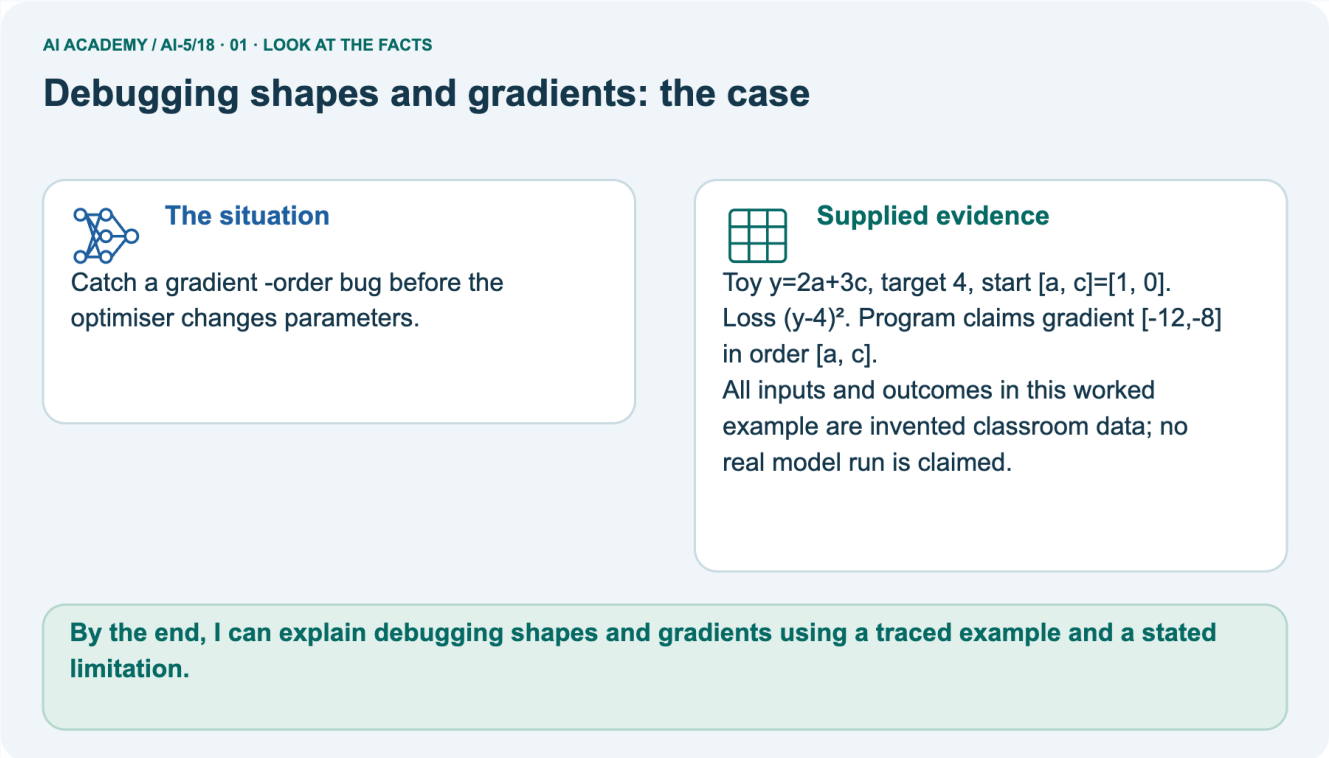
Drafts stay in this browser when storage is available; clear drafts on a shared device.

STUDENT LESSON · 18 OF 18

Visual concept journey

> Explain each Debugging shapes and gradients graphic using its labels, deciding evidence and relevant limitation.

01. Debugging shapes and gradients: the case



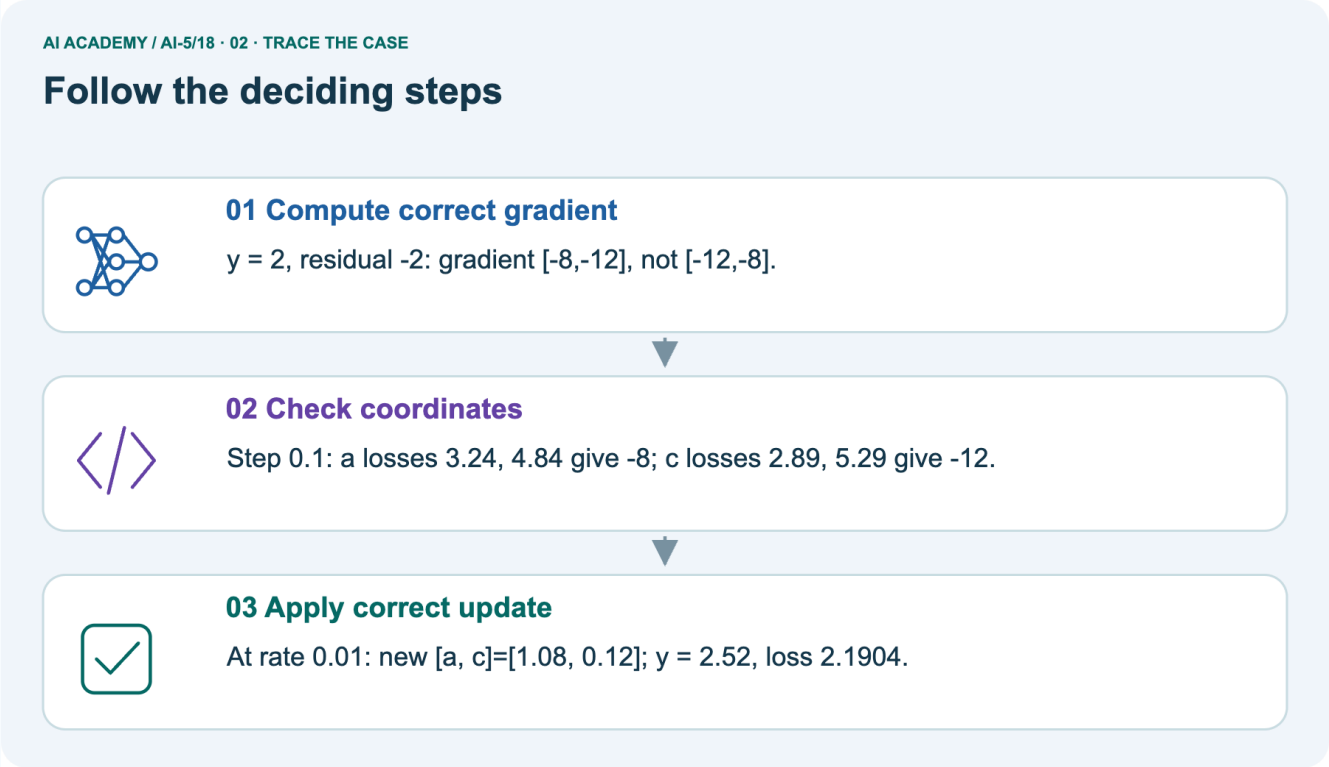
Follow the labelled picture with the matching explanation. Open the vector diagram

Catch a gradient -order bug before the optimiser changes parameters. Supplied facts: Toy y=2a+3c, target 4, start [a, c]=[1, 0]. Loss (y-4)². Program claims gradient [-12,-8] in order [a, c].

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Compute correct gradient: y = 2, residual -2: gradient [-8,-12], not [-12,-8]. → Check coordinates: Step 0.1: a losses 3.24, 4.84 give -8; c losses 2.89, 5.29 give -12. → Apply correct update: At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

Point to each numbered stage and explain which recorded **input** it uses.

03. Repair from the first disagreement

AI ACADEMY / AI-5/18 · 03 · INSPECT THE LESSON MODEL

Repair from the first disagreement

Trace stage for x [1 2]	Expected result	Submitted result
Input and dimensions	x length 2 W 3 by 2 b length 3	Dimensions match
Scores before ReLU	[1, 2, 0]	[1, 2, -1]
Hidden activations	[1, 2, 0]	[1, 2, 0]

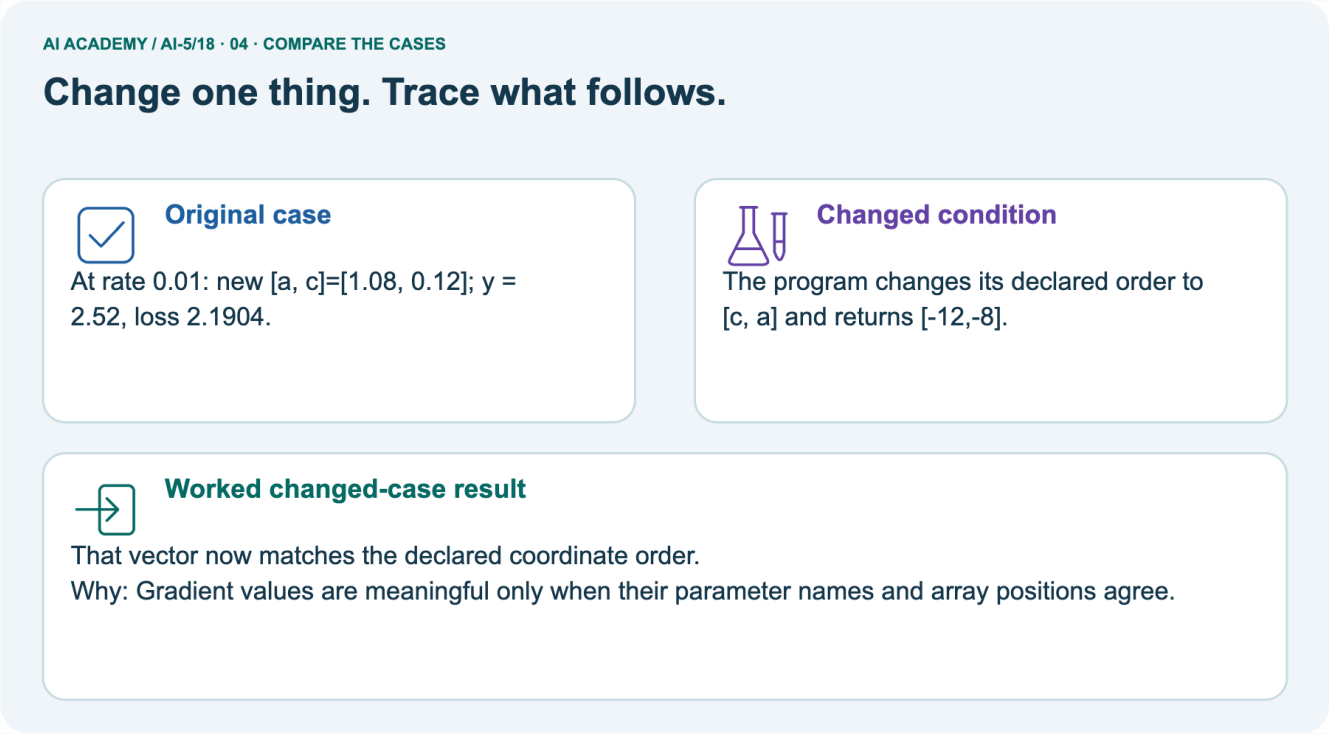
The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At x = [3, 1], the same omission changes the third activation from 3 to 2.

Follow the labelled picture with the matching explanation. Open the vector diagram

Repair from the first disagreement. **Input** and dimensions | x length 2 W 3 by 2 b length 3 | Dimensions match. Scores before ReLU | [1, 2, 0] | [1, 2, -1]. Hidden activations | [1, 2, 0] | [1, 2, 0]

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



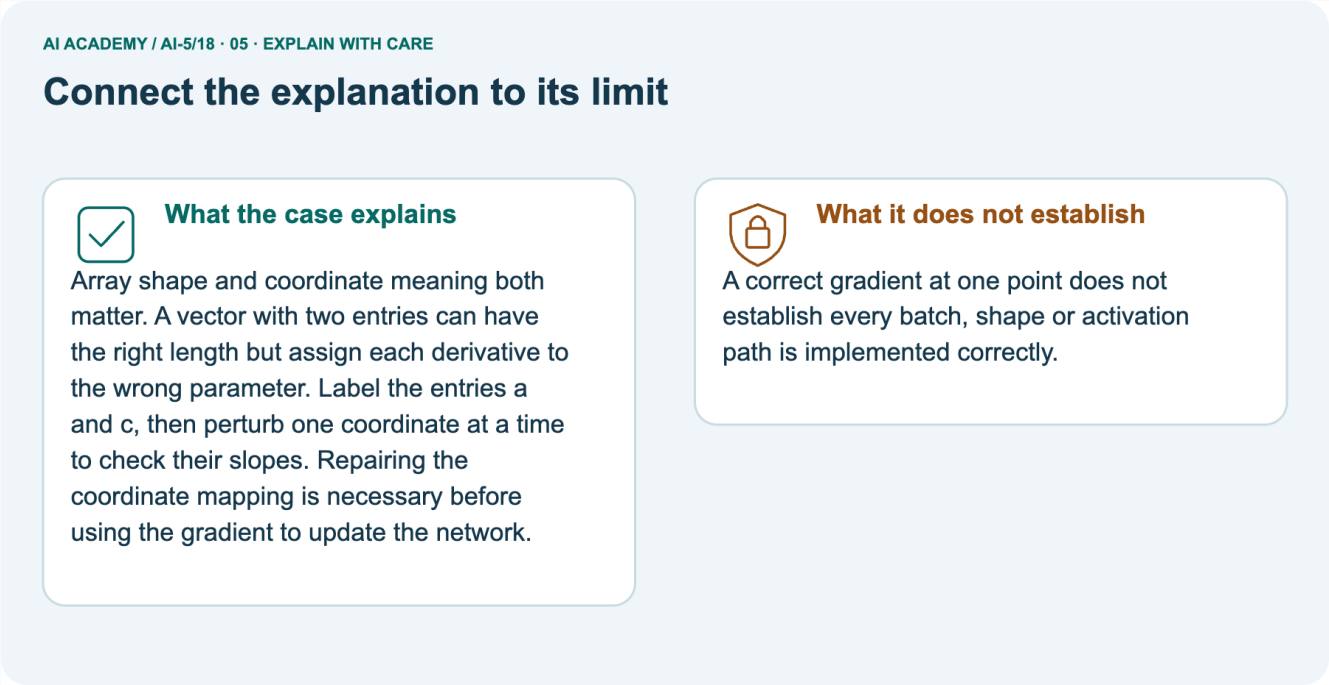
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904. Changed condition: The program changes its declared order to [c, a] and returns [-12,-8].

Expected result: That vector now matches the declared coordinate order. Reason: Gradient values are meaningful only when their parameter names and array positions agree.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

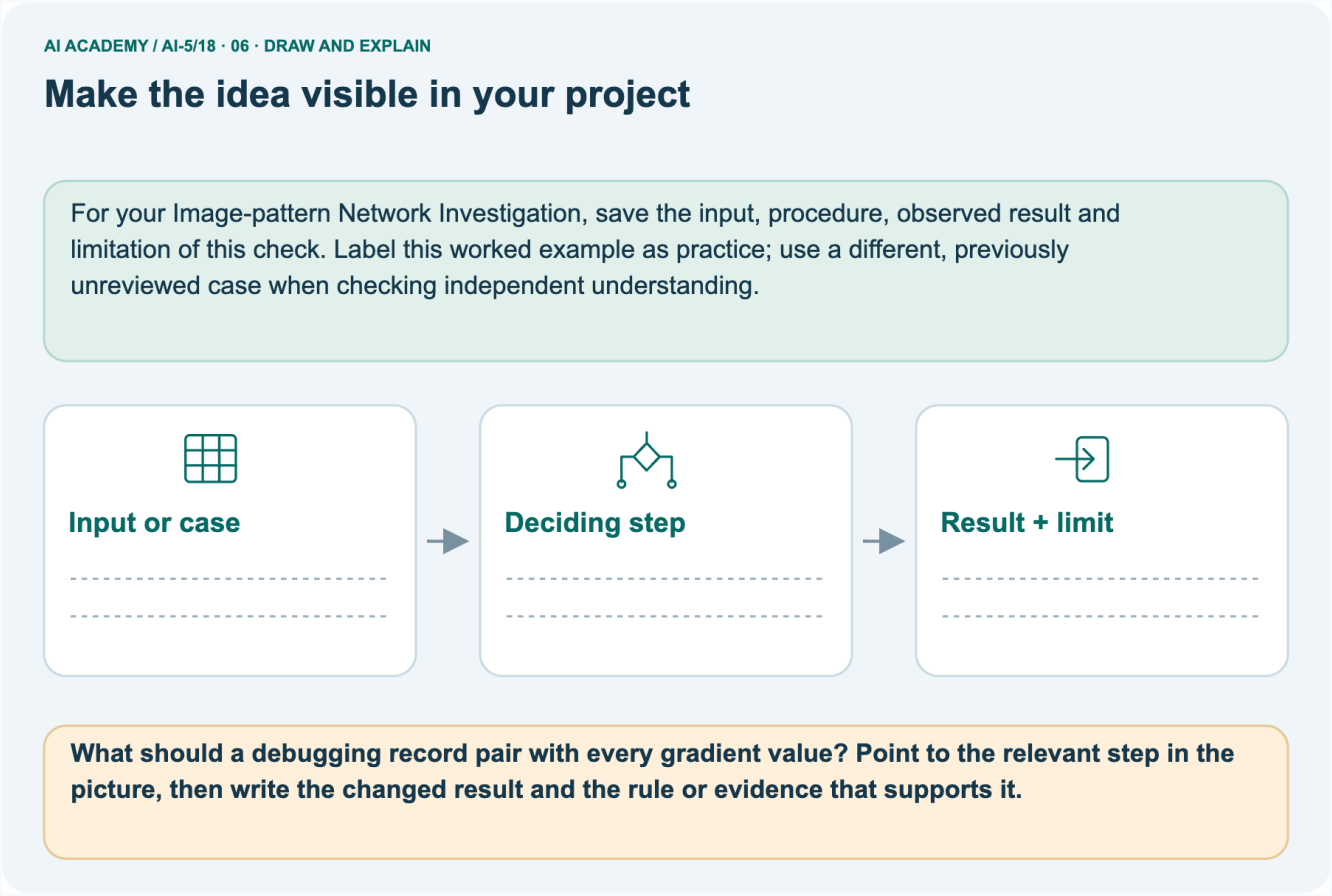
Array **shape** and coordinate meaning both matter. A vector with two entries can have the right length but assign each derivative to the wrong parameter.

Label the entries a and c, then perturb one coordinate at a time to check their slopes. Repairing the coordinate mapping is necessary before using the gradient to update the network.

Limit: A correct gradient at one point does not establish every batch, **shape** or activation path is implemented correctly.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.